

UAS MINI PROJECT INDIVIDU

Communication Protocol

Reliability API: Rate Limiting & Load Balancing Testing (P10)

Nama	Nacwa Julianti
NIM	25120500019
Kelas	Semester 2 / ComP2
Mata Kuliah	Communication Protocol
Dosen Pengampu	Haikal Shiddiq, S.Kom., M.T.
Link GitHub	https://github.com/nacwajulianti55-tech/uas-commprotocol-nacwajulianti-25120500019/tree/main
Link YouTube	https://youtu.be/CtzQpca5ZOE

1. Problem Statement, Tujuan, dan Alasan Pemilihan Use Case

Problem Statement

API modern yang diakses banyak client secara bersamaan rentan terhadap dua masalah reliability klasik: (1) satu client dapat mengirim request berlebihan hingga membebani server (perlu rate limiting), dan (2) satu server tunggal dapat menjadi titik kegagalan tunggal jika seluruh traffic dibebankan padanya (perlu load balancing). Project ini menguji bagaimana kedua mekanisme itu diimplementasikan dan dibuktikan secara teknis pada demo app P10 Reliability API.

Tujuan Mini Project

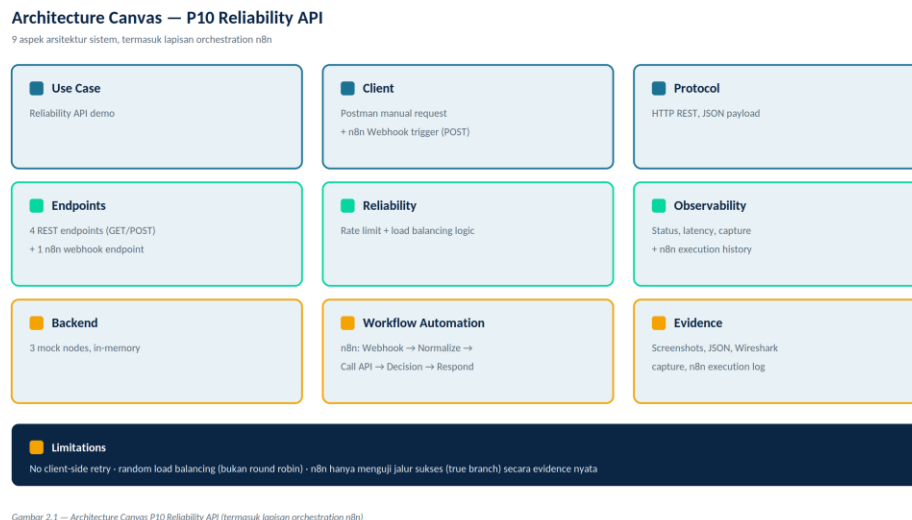
- Membuktikan mekanisme rate limiting (fixed window counter) bekerja melalui pengujian berulang.
- Membuktikan mekanisme load balancing (distribusi ke beberapa backend node) melalui variasi response.
- Mendokumentasikan reliability dan error handling melalui skenario success dan failure.
- Membuktikan observability melalui capture traffic Wireshark dan response time Postman.

Alasan Memilih Use Case

Use case yang dipilih adalah **Demo App P10 Reliability API**, karena melanjutkan praktikum yang sudah dikerjakan sebelumnya dan menyediakan endpoint yang secara eksplisit didesain untuk mendemonikan konsep rate limiting dan load balancing dengan output berupa status code, response time, dan teaching point yang jelas.

2. Architecture Canvas dan Data Flow Diagram

Architecture canvas berikut merangkum sembilan aspek arsitektur sistem, digambar sendiri berdasarkan hasil pengujian endpoint P10 Reliability API.

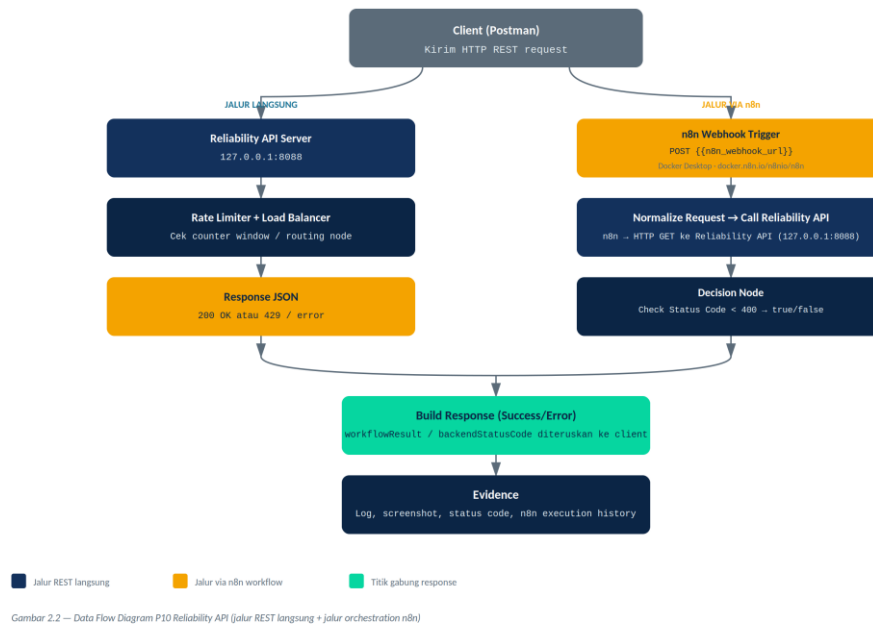


Gambar 2.1 — Architecture Canvas P10 Reliability API

Data flow diagram berikut menunjukkan alur lengkap dari client mengirim request hingga evidence tercatat, sesuai pola client → protocol → service/workflow → storage/log/evidence.

Data Flow Diagram — P10 Reliability API

Dua jalur client: REST langsung, dan via n8n workflow automation (webhook)



Gambar 2.2 — Data Flow Diagram P10 Reliability API

3. Protocol Selection

Protocol utama yang digunakan adalah **HTTP REST** dengan format payload JSON. Berikut alasan teknis pemilihannya:

- REST over HTTP dipilih karena rate limiting dan load balancing lazim diimplementasikan di level HTTP/API gateway; status code (200, 429) menjadi bahasa standar untuk mengomunikasikan kondisi tersebut ke client.
- Interaksi bersifat request-response sesaat (client meminta, server menjawab), bukan koneksi persisten atau event streaming, sehingga REST lebih sesuai dibanding WebSocket atau MQTT untuk use case ini.
- Sifat stateless REST relevan karena setiap request rate-limited dievaluasi independen berdasarkan window waktu (10 detik), bukan berdasarkan sesi koneksi yang terbuka.

Base URL : **http://127.0.0.1:8088**

4. Endpoint Webhook n8n

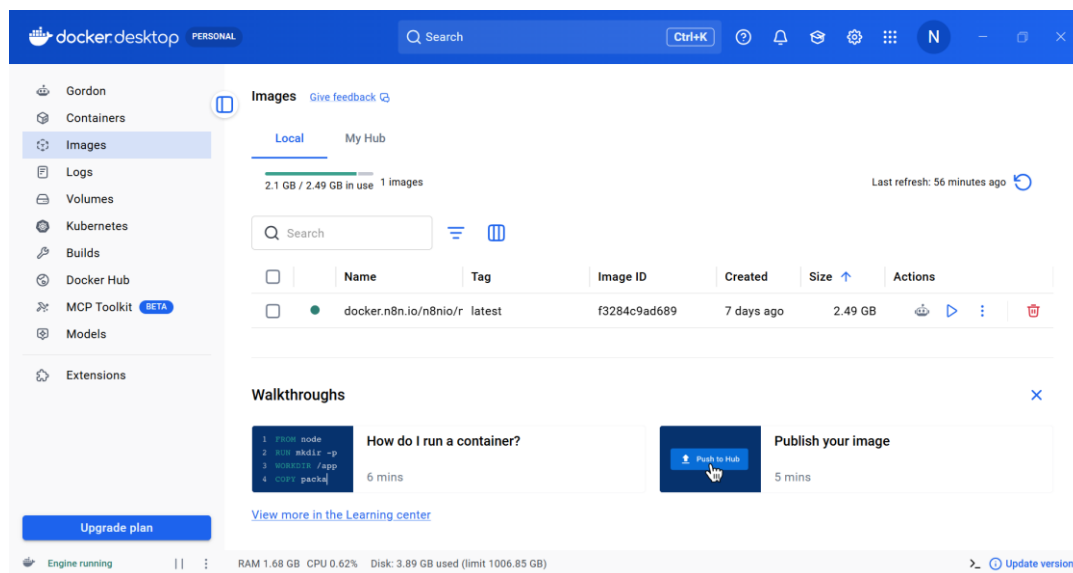
n8n merupakan platform **workflow automation** yang digunakan untuk menghubungkan aplikasi atau API melalui alur kerja otomatis. Pada mini project ini, n8n digunakan untuk menerima request dari Postman melalui **Webhook**, memproses workflow yang telah dikonfigurasi, kemudian mengembalikan response dalam format **JSON** dengan status **HTTP 200 OK**. Penggunaan n8n menjadi pelengkap implementasi **Communication Protocol** melalui integrasi berbasis HTTP.

4.1 Integrasi Workflow Automation Menggunakan n8n

4.1.1 Docker Desktop

Pada mini project ini, Docker Desktop digunakan sebagai platform untuk menjalankan layanan n8n dalam bentuk container. Dengan menggunakan Docker, proses instalasi dan pengelolaan n8n menjadi lebih mudah karena seluruh kebutuhan aplikasi telah dikemas ke dalam sebuah image. Setelah Docker Engine aktif, image `docker.n8n.io/n8nio/n8n:latest` dijalankan sehingga layanan n8n dapat diakses melalui browser dan digunakan untuk membuat workflow automation.

Penggunaan Docker Desktop pada mini project ini bertujuan untuk menyediakan lingkungan yang stabil dalam menjalankan n8n tanpa perlu melakukan instalasi langsung pada sistem operasi. Selain itu, Docker juga memudahkan proses menjalankan kembali layanan apabila diperlukan selama proses pengujian.

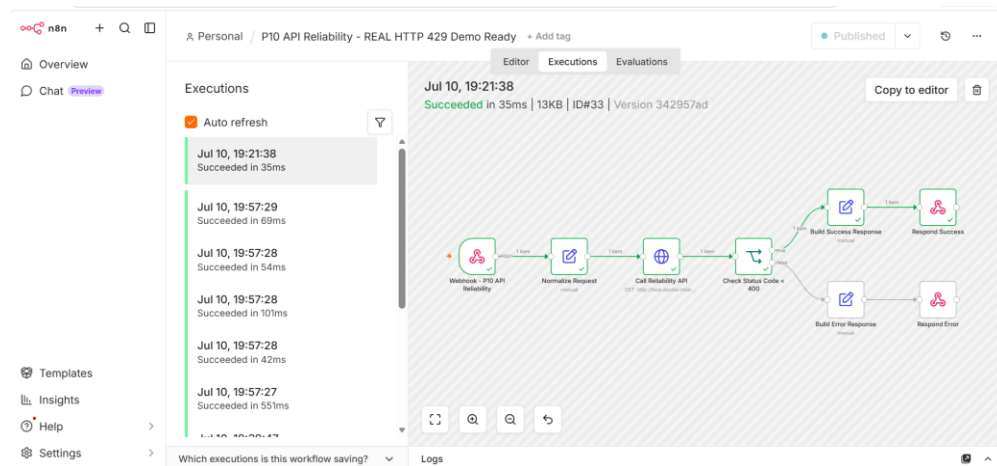


Gambar 4.1 Docker Desktop yang digunakan untuk menjalankan image n8n.

Docker Desktop berada dalam kondisi Engine running yang menandakan layanan Docker telah aktif. Terlihat image `docker.n8n.io/n8nio/n8n:latest` telah tersedia pada Docker Desktop dan siap digunakan untuk menjalankan workflow n8n. Kondisi ini menunjukkan bahwa lingkungan yang digunakan untuk menjalankan workflow automation telah berhasil disiapkan sebelum dilakukan proses pengujian menggunakan Postman.

4.1.2 Workflow n8n

Workflow pada n8n digunakan sebagai media **workflow automation** yang menghubungkan Postman dengan Demo App P10 Reliability API. Workflow ini menerima request dari Postman melalui **Webhook**, kemudian meneruskan request ke endpoint Reliability API untuk diproses. Setelah API memberikan response, workflow akan melakukan pengecekan status response dan menentukan apakah hasil tersebut termasuk **berhasil (success)** atau **gagal (error)** sebelum mengirimkan response kembali kepada client.



Gambar 4.2 Workflow Automation menggunakan n8n

Workflow dimulai dari node **Webhook – P10 API Reliability**. Node ini berfungsi sebagai titik masuk (endpoint) yang menerima request **HTTP POST** dari aplikasi Postman. Setiap request yang diterima akan memicu workflow agar dijalankan secara otomatis.

Setelah request diterima, data diteruskan ke node **Normalize Request**. Node ini digunakan untuk menyesuaikan atau menyusun kembali format data request sehingga sesuai dengan kebutuhan sebelum dikirim ke backend API.

Selanjutnya request dikirim ke node **Call Reliability API**. Pada tahap ini, n8n melakukan request **HTTP GET** ke Demo App P10 Reliability API yang berjalan pada server lokal. Node ini bertugas mengambil response dari endpoint Reliability API yang sedang diuji.

Response dari API kemudian diproses oleh node **Check Status Code < 400**. Node ini berfungsi sebagai percabangan logika (decision node) untuk memeriksa nilai **HTTP Status Code** yang diterima dari backend. Apabila nilai status code kurang dari **400**, workflow akan mengikuti jalur **true** yang menandakan request berhasil diproses. Sebaliknya, apabila status code bernilai **400** atau lebih, workflow akan mengikuti jalur **false** yang menandakan terjadi kesalahan.

Pada hasil eksekusi yang ditampilkan pada gambar, workflow mengikuti jalur **true** sehingga proses diteruskan ke node **Build Success Response**. Node ini digunakan untuk menyusun data response dalam format **JSON** yang berisi informasi hasil pengujian, seperti status workflow, status backend, serta informasi pendukung lainnya.

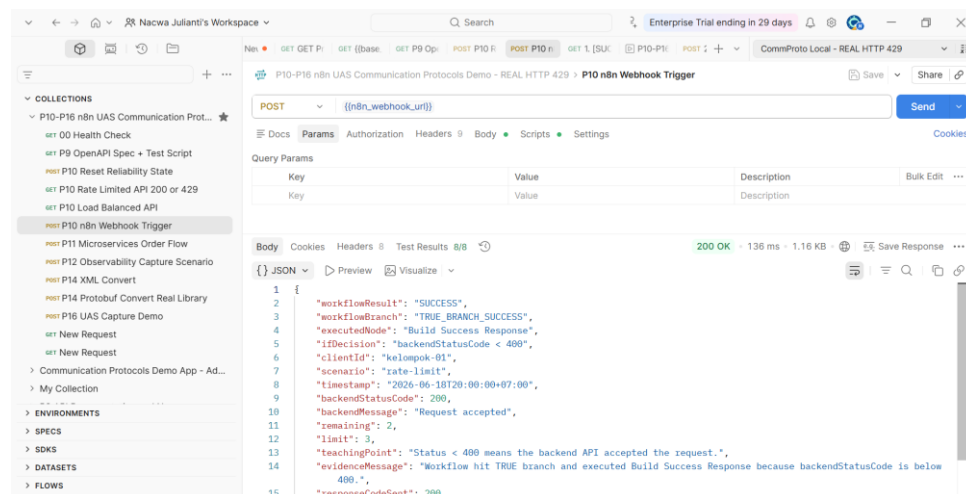
Setelah response selesai dibentuk, workflow dilanjutkan ke node **Respond Success** yang bertugas mengirimkan response kembali kepada Postman. Hal ini menunjukkan bahwa seluruh proses workflow berhasil dijalankan tanpa terjadi kesalahan.

Pada bagian kiri tampilan juga terlihat beberapa riwayat **Execution** dengan status **Succeeded**, yang menunjukkan bahwa workflow telah berhasil dijalankan beberapa kali. Eksekusi terakhir berlangsung pada **10 Juli pukul 19.21.38** dengan waktu proses sekitar **35 ms**, menandakan bahwa komunikasi antara Postman, n8n, dan Reliability API berlangsung dengan cepat dan berhasil.

Berdasarkan hasil tersebut, dapat disimpulkan bahwa workflow n8n berhasil mengotomatisasi proses komunikasi antara client dan backend API, mulai dari menerima request, memanggil endpoint Reliability API, melakukan evaluasi terhadap status response, hingga mengirimkan hasil akhir kepada client secara otomatis.

4.1.3 Pengujian Webhook Menggunakan Postman

Pengujian workflow dilakukan menggunakan aplikasi **Postman** dengan metode **POST** yang dikirim menuju endpoint **Webhook** milik n8n. Setelah request diterima, workflow akan diproses sesuai konfigurasi yang telah dibuat, kemudian n8n mengembalikan response kepada client dalam format JSON.



Gambar 4.3 Pengujian Webhook n8n menggunakan Postman.

request berhasil diproses oleh workflow n8n dengan menghasilkan status **HTTP 200 OK**. Response menunjukkan nilai **workflowResult** sebesar **SUCCESS**, yang menandakan workflow berhasil dijalankan tanpa mengalami kesalahan. Workflow juga masuk ke jalur **TRUE_BRANCH_SUCCESS** dan mengeksekusi node **Build Success Response** sebelum mengirimkan response kembali ke Postman.

Selain itu, response menampilkan **backendStatusCode** sebesar **200** dengan pesan **Request accepted**, yang menunjukkan bahwa request berhasil diterima dan diproses oleh workflow. Informasi **remaining** dan **limit** juga ditampilkan sebagai bagian dari data response yang menunjukkan status mekanisme rate limiting pada proses pengujian. Hasil tersebut membuktikan bahwa integrasi antara **Postman**, **Docker Desktop**, dan **n8n** berjalan dengan baik sehingga workflow automation dapat menerima request, memproses data, dan mengembalikan response sesuai dengan konfigurasi yang telah dibuat.

5. Endpoint / Action Design

No	Method	Path	Headers	Request Body	Expected Status
1	GET	/api/reliability/rate-limited	Content-Type: application/json	-	200 / 429
2	GET	/api/reliability/load-balanced	Content-Type: application/json	-	200
3	POST	/api/reliability/simulate-load	Content-Type: application/json	{"requests": N}	error
4	POST	/api/reliability/reset	Content-Type: application/json	- (atau {})	200
5	POST	{{n&n_webhook_url}}	Content-Type: application/json	{"clientId": "kelompok-01", "scenario": "rate-limit", "timestamp": "..."} 	200

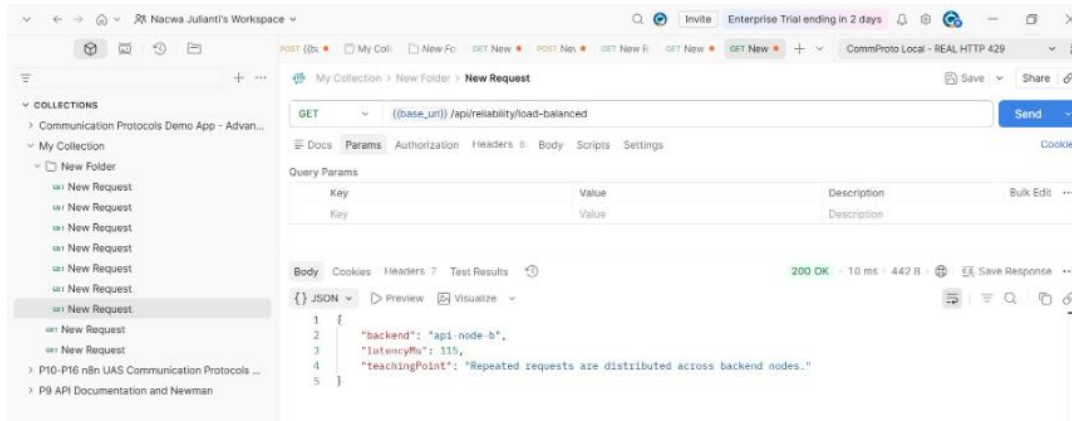
Detail response tiap endpoint (diambil dari hasil pengujian nyata di Postman):

Endpoint	Response Body (contoh nyata)
GET rate-limited (429)	<pre>{ "error": "Too Many Requests", "limit": 3, "windowSeconds": 10, "retryAfterSeconds": 1 }</pre>
GET load-balanced	<pre>{ "backend": "api-node-a/b/c", "latencyMs": 80-150, "teachingPoint": "..."} </pre>
POST simulate-load	<pre>{ "requests": "banyak" } </pre> -> tidak divalidasi, menyebabkan error koneksi
POST reset	<pre>{ "message": "Reliability demo state reset" } </pre>
POST n8n Webhook	<pre>{ "workflowResult": "SUCCESS", "workflowBranch": "TRUE_BRANCH_SUCCESS", "executedNode": "Build Success Response", "backendStatusCode": 200, "remaining": 2, "limit": 3, "teachingPoint": "..."} </pre>

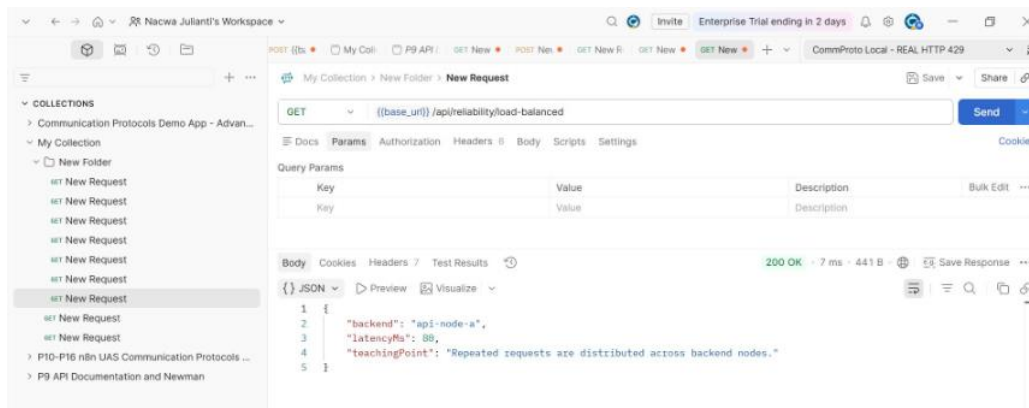
6. Evidence Success Scenario

6.1 Success — Load Balancing (3 backend node berbeda)

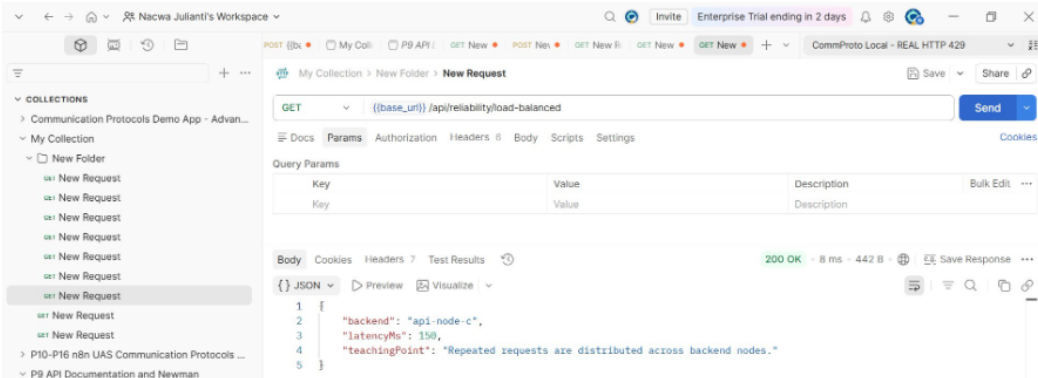
Tiga kali percobaan pada endpoint **GET /api/reliability/load-balanced** menunjukkan server mendistribusikan request ke tiga backend node berbeda secara bergantian, membuktikan mekanisme load balancing berjalan.



Gambar 6.1 — Response 200 OK, backend: api-node-b, latency 115ms



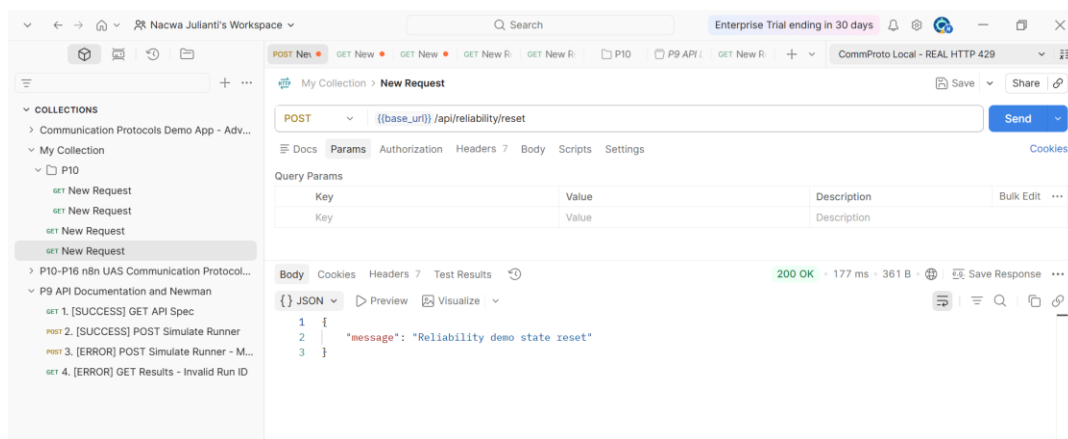
Gambar 6.2 — Response 200 OK, backend: api-node-a, latency 80ms



Gambar 6.3 — Response 200 OK, backend: api-node-c, latency 150ms

6.2 Success — Reset Reliability State

Endpoint **POST /api/reliability/reset** berhasil mengembalikan status 200 OK dengan pesan konfirmasi reset, menunjukkan server tetap berfungsi normal untuk melanjutkan pengujian berikutnya.

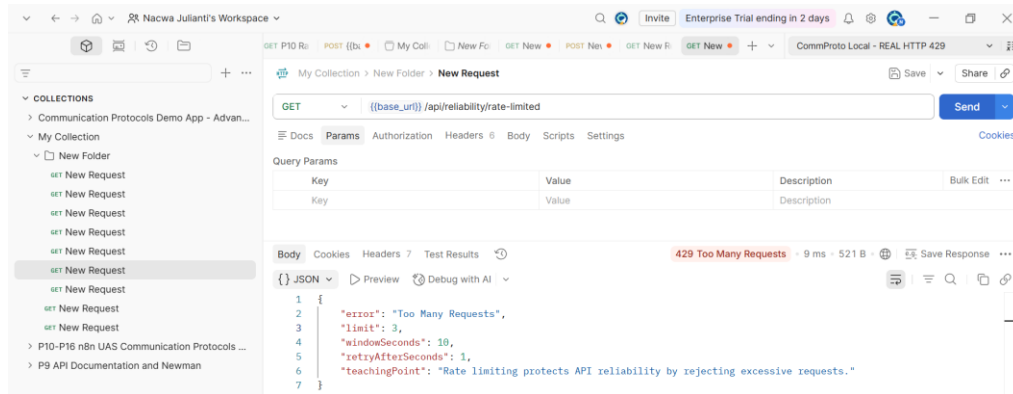


Gambar 6.4 — Response 200 OK, state reliability berhasil direset

7. Evidence Failure / Error Scenario

7.1 Failure — Rate Limit Exceeded

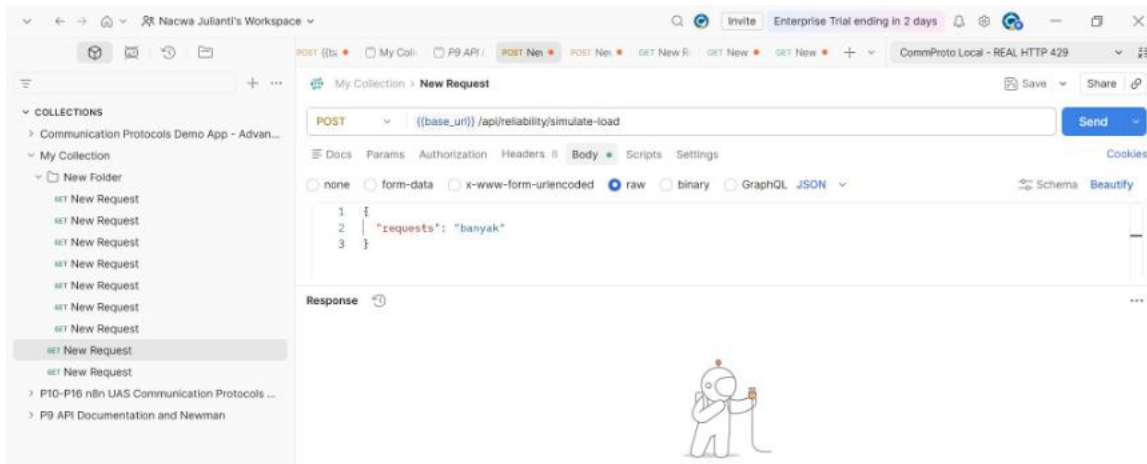
Setelah tiga request berhasil dalam window 10 detik, request keempat pada endpoint yang sama ditolak dengan status **429 Too Many Requests**. Server juga menyertakan field `retryAfterSeconds` yang memberi tahu client kapan boleh mencoba lagi.



Gambar 7.1 — Response 429 Too Many Requests dengan `retryAfterSeconds`

7.2 Failure — Input Tidak Divalidasi (Simulate Load)

Ketika field requests diisi dengan tipe data yang salah (string "banyak", bukan angka), server tidak melakukan validasi tipe data sehingga terjadi kegagalan koneksi (socket hang up) alih-alih mengembalikan error response terstruktur seperti 400 Bad Request. Ini menunjukkan celah pada error handling aplikasi.



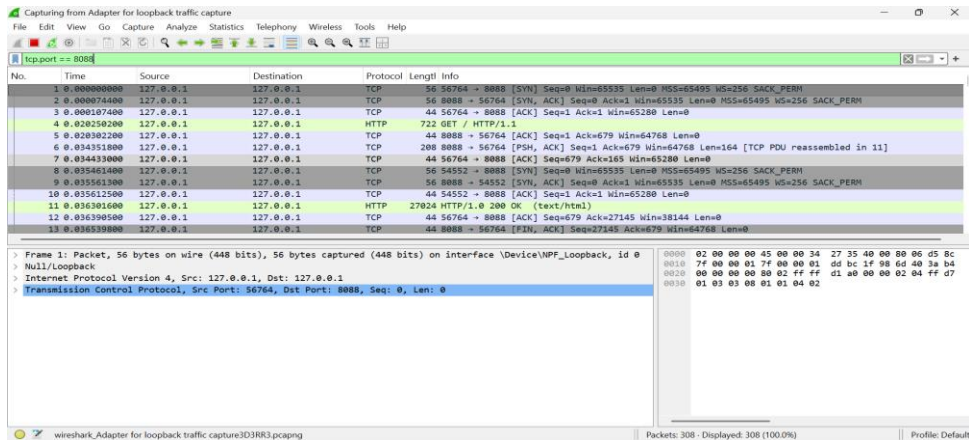
Gambar 7.2 — Request body dengan tipe data salah pada endpoint `simulate-load`

8. Observability dan Traffic Evidence

8.1 Wireshark — Full HTTP Request/Response Cycle

Capture pada interface loopback dengan filter `tcp.port == 8088` menangkap siklus lengkap komunikasi:

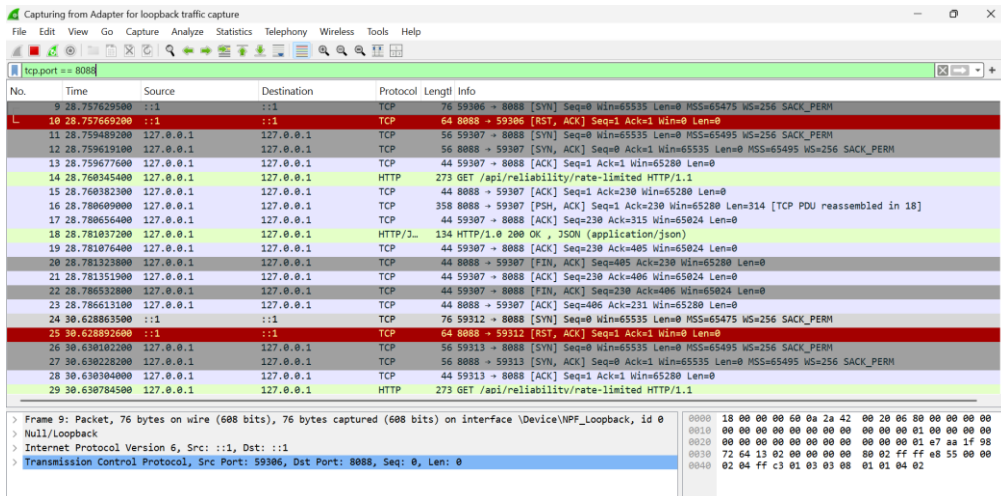
TCP three-way handshake (SYN, SYN-ACK, ACK), pengiriman HTTP GET, response 200 OK yang di-reassemble karena ukurannya melebihi satu segmen TCP, hingga penutupan koneksi dengan FIN-ACK.



Gambar 8.1 — Wireshark: TCP handshake, HTTP GET, response 200 OK, FIN-ACK

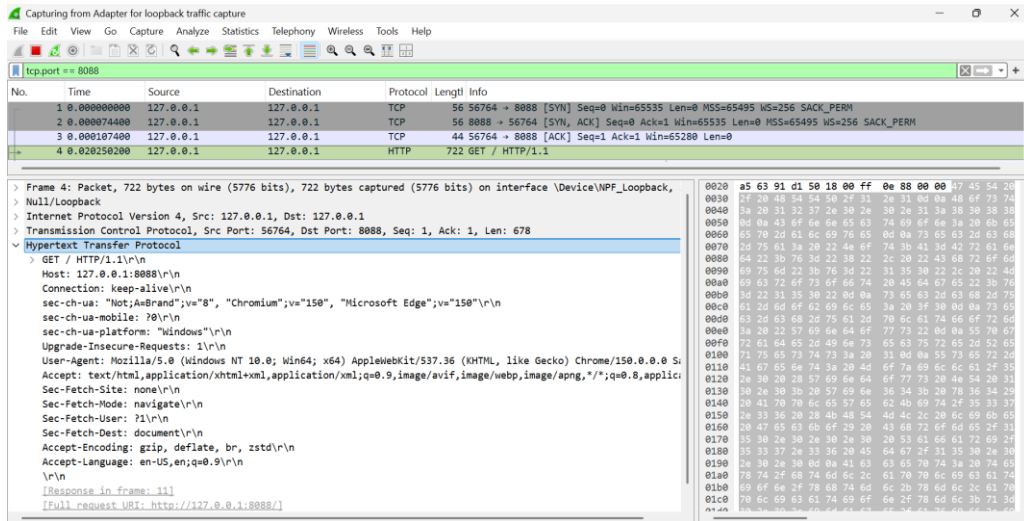
8.2 Wireshark — Request Nyata dari Postman ke Endpoint Rate-Limited

Capture kedua menangkap request langsung dari Postman ke endpoint `/api/reliability/rate-limited` dengan response Content-Type `application/json`. Terlihat juga percobaan koneksi awal via IPv6 (::1) yang ditolak server (RST, ACK) karena server hanya listen pada IPv4, sebelum sistem otomatis fallback ke 127.0.0.1 yang berhasil — perilaku dual-stack networking standar mirip mekanisme Happy Eyeballs.



Gambar 8.2 — Wireshark: request Postman ke rate-limited, termasuk fallback IPv6 ke IPv4

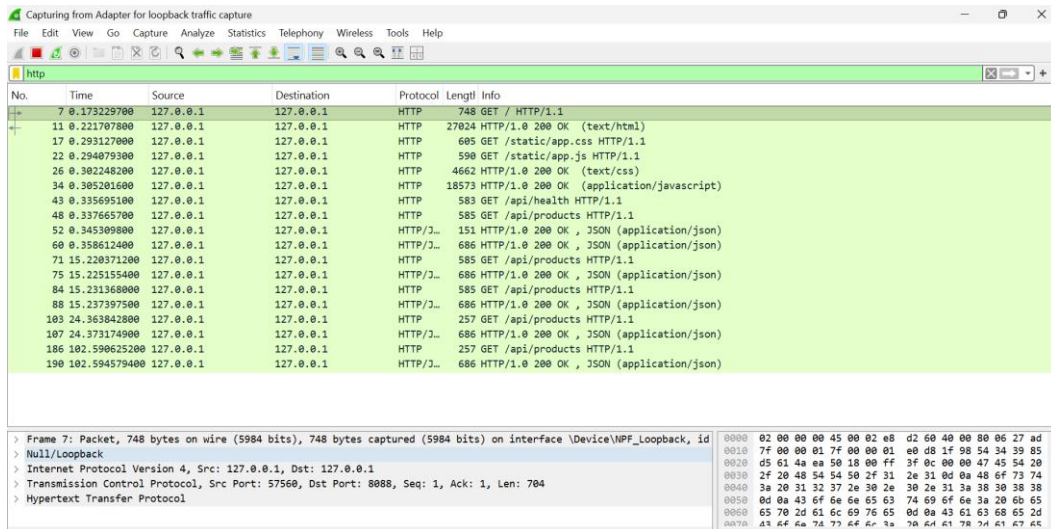
8.3 Wireshark — Detail HTTP Header



Gambar 8.3 — Detail header HTTP request (User-Agent, Host, Accept, dll)

8.4 Wireshark — Traffic Filtered by HTTP (Multi-Request)

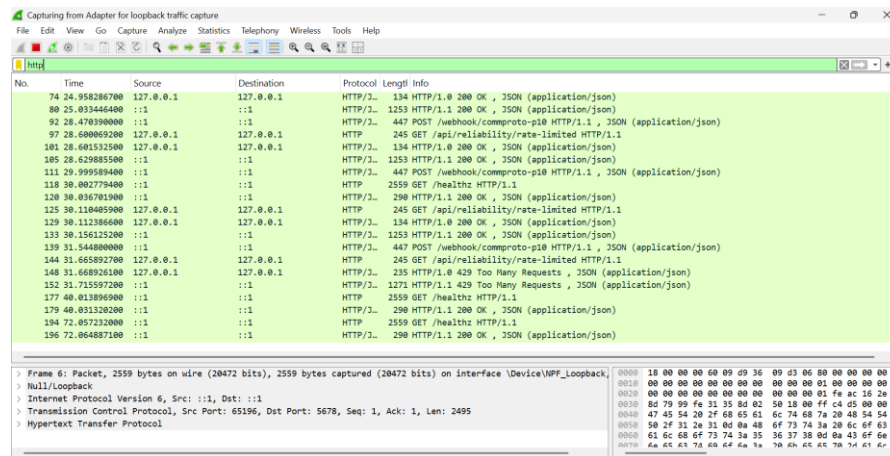
Capture tambahan dengan filter http menunjukkan beberapa endpoint lain yang sempat diakses selama sesi browsing demo app, termasuk static asset dan endpoint API lain, memberikan gambaran traffic HTTP secara lebih luas.



Gambar 8.4 — Wireshark filtered by http, menampilkan beberapa request berurutan

8.5 Wireshark - Traffic/Protocol Observation

Pengamatan traffic jaringan dilakukan menggunakan **Wireshark** dengan filter **HTTP** untuk melihat komunikasi antara **Postman**, **n8n**, dan **Demo API Reliability**. Hasil capture menunjukkan bahwa komunikasi menggunakan protokol **HTTP**.



Gambar 8.5 — Wireshark yang menunjukkan request POST ke Webhook n8n serta response HTTP 200 OK dan HTTP 429 Too Many Requests.

9. Testing Result dan Reliability Analysis

9.1 Ringkasan Hasil Testing

Jenis	Endpoint	Hasil
Success 1	GET load-balanced (3x)	200 OK, backend a/b/c berbeda
Success 2	POST reset	200 OK
Failure 1	GET rate-limited (ke-4x)	429 Too Many Requests
Failure 2	POST simulate-load (tipe salah)	Socket hang up / connection error

9.2 Analisis Reliability

- **Rate limiting** — Server menerapkan pola fixed window counter (limit 3 request per 10 detik). Field `retryAfterSeconds` pada response 429 menunjukkan server memberi tahu client kapan boleh mencoba lagi, mendukung strategi retry di sisi client.
- **Load balancing** — Berdasarkan urutan backend yang muncul (b, a, c) pada tiga percobaan berturut-turut, polanya tidak mengikuti round robin yang konsisten maupun least-latency, mengindikasikan algoritma random selection antar tiga backend node.
- **Error contract** — Response error konsisten menyertakan field `message/error`, `limit`, dan `windowSeconds`, memudahkan client memahami penyebab kegagalan tanpa harus menebak.

- **Gap reliability** — Endpoint simulate-load tidak memvalidasi tipe data pada field requests, menyebabkan koneksi terputus alih-alih mengembalikan error terstruktur. Ini adalah temuan penting yang menunjukkan reliability aplikasi belum sepenuhnya graceful terhadap input tidak terduga.
- **Partial resilience** — Meski satu request bermasalah menyebabkan koneksi terputus, server tetap dapat melayani request berikutnya (dibuktikan endpoint reset yang tetap berhasil setelahnya), menunjukkan kegagalan bersifat isolated per-request, bukan crash keseluruhan sistem.
- **Workflow automation (n8n)** — Pengujian webhook n8n menunjukkan workflow berhasil mengorquestrasi panggilan ke Reliability API dan mengevaluasi hasilnya dengan benar: saat backendStatusCode bernilai 200, decision node Check Status Code < 400 mengambil jalur TRUE_BRANCH_SUCCESS dan mengeksekusi node Build Success Response, menghasilkan workflowResult: "SUCCESS" dengan seluruh 8 dari 8 test assertion Postman lolos. Response akhir yang diteruskan ke client turut menyertakan konteks reliability asli dari backend (remaining, limit, teachingPoint), menunjukkan bahwa lapisan orchestration n8n tidak menyembunyikan informasi reliability, melainkan meneruskannya secara transparan kepada client.

10. Kesimpulan, Limitation, Improvement, dan Reflection Pembelajaran

Kesimpulan

Project ini berhasil membuktikan bahwa demo app P10 Reliability API menerapkan dua mekanisme reliability utama, yaitu rate limiting dan load balancing, melalui pengujian nyata menggunakan Postman. Rate limiting terbukti bekerja dengan pola fixed window counter (limit 3 request per 10 detik), ditunjukkan oleh response 200 OK pada tiga request pertama dan 429 Too Many Requests pada request keempat. Load balancing juga terbukti berjalan, dikonfirmasi oleh distribusi request ke tiga backend node berbeda (api-node-a, api-node-b, api-node-c) dengan latency yang bervariasi. Seluruh syarat minimal soal UAS telah terpenuhi, meliputi pengujian terhadap empat endpoint, tiga skenario sukses, dua skenario gagal, architecture canvas, data flow diagram, serta evidence traffic Wireshark.

Limitation

Beberapa keterbatasan ditemukan selama pengujian. Pertama, pola distribusi load balancer tidak mengikuti algoritma round robin yang konsisten maupun least-latency, sehingga algoritma pastinya tidak dapat dipastikan sepenuhnya tanpa akses ke source code aplikasi. Kedua, endpoint simulate-load belum memiliki validasi tipe data pada field requests, sehingga input yang salah tipe menyebabkan koneksi terputus (socket hang up) alih-alih mengembalikan error response yang terstruktur. Ketiga, pengujian ini masih dilakukan secara manual di Postman tanpa mekanisme retry otomatis di sisi client, meskipun server sebenarnya sudah menyediakan informasi retryAfterSeconds yang dapat dimanfaatkan untuk hal tersebut.

Improvement

Beberapa perbaikan yang dapat diusulkan untuk pengembangan lebih lanjut antara lain menambahkan validasi tipe data pada seluruh endpoint yang menerima input POST, agar server tidak mengalami unhandled exception

saat menerima data yang tidak sesuai format. Selain itu, mekanisme retry otomatis di sisi client dapat diimplementasikan dengan memanfaatkan header `retryAfterSeconds` yang sudah tersedia pada response 429. Terakhir, penambahan logging terstruktur di sisi server akan mempermudah proses observability tanpa harus selalu bergantung pada capture Wireshark secara manual.

Reflection Pembelajaran

Melalui pengerjaan project ini, saya belajar bahwa reliability sebuah API tidak hanya soal endpoint yang berjalan dengan benar, tetapi juga soal bagaimana sistem menangani kondisi tidak ideal seperti input yang salah atau beban berlebih. Menemukan celah error handling pada endpoint `simulate-load` justru menjadi pembelajaran paling berharga, karena menunjukkan bahwa pengujian failure scenario sama pentingnya dengan pengujian success scenario dalam memahami kualitas sebuah sistem secara menyeluruh.